

Convenções de Uso do Git

Projeto DeuBranco

Documento complementar ao README — versão 1.0 — 19/06/2026

1. Objetivo

Este documento define as regras de uso do Git adotadas pela equipe do projeto DeuBranco a partir desta versão, cobrindo padrão de mensagens de commit, estratégia de branches, organização de pastas entre documentação e código, e uso do arquivo `.gitignore`. O objetivo é facilitar o trabalho colaborativo entre os três integrantes da equipe e tornar o histórico do repositório mais legível e rastreável.

Estas regras passam a valer a partir da data de publicação deste documento e aplicam-se a todos os commits futuros. Elas não invalidam, anulam ou exigem reescrita de nenhum commit já existente no histórico do repositório: o histórico anterior é preservado como está, como registro legítimo da evolução do projeto até este ponto.

2. Padrão de mensagens de commit

A equipe adota como referência o padrão de commits semânticos descrito no repositório `iuricode/padrees-de-commits` (baseado em Conventional Commits), disponível em:

<https://github.com/iuricode/padrees-de-commits>

A partir de agora, toda mensagem de commit deve seguir a estrutura abaixo:

tipo: descrição breve no infinitivo ou imperativo, em português

Exemplos de mensagens corretas dentro do novo padrão:

- feat: adiciona autenticação JWT para jogadores
- fix: corrige relação extend incorreta entre RF006 e RF008
- docs: atualiza diagrama de pacotes
- chore: adiciona extensões de build ao gitignore

2.1 Tabela de tipos

Tipo	Quando usar
feat	Inclusão de uma nova funcionalidade no sistema.
fix	Correção de um problema (bug) identificado no código.
docs	Alterações em documentação (README, diagramas, dicionários, especificações). Não envolve código.
refactor	Reestruturação de código existente sem alterar o comportamento observável (ex.: melhoria de performance, reorganização de classes).
style	Alterações de formatação, indentação, espaçamento ou convenções de nomenclatura que não afetam a lógica.

test	Criação, alteração ou remoção de testes automatizados.
build	Mudanças em arquivos de build, dependências ou configuração do ambiente de desenvolvimento.
chore	Tarefas de manutenção que não alteram código de produção nem documentação (ex.: ajuste no .gitignore, configuração de ferramentas).

2.2 Compatibilidade com o histórico existente

Uma parte dos commits já realizados no repositório segue parcialmente este padrão (ex.: "docs: readme completo", "feat: role para jogadores e admins"), enquanto outra parte foi feita antes da formalização desta regra e por isso não traz o prefixo de tipo (ex.: "diagrama de classes", "diagrama de classes de dominio", "Upload de print com padrões"). Esses commits permanecem válidos: eles refletem o estado do projeto em commits anteriores a esta convenção e não serão alterados, reescritos (rebase/amend) ou removidos do histórico. A regra de padronização é prospectiva, ou seja, aplica-se aos commits feitos a partir da adoção deste documento.

Caso seja necessário referenciar um desses commits antigos em discussões futuras (ex.: revisão de diagrama de classes), a equipe pode usar um commit novo no padrão correto para registrar a complementação, por exemplo:

```
docs: complementa diagrama de classes com atributos derivados
```

2.3 Boas práticas adicionais de commit

- Escrever mensagens curtas e diretas na primeira linha (idealmente até 50-60 caracteres); detalhes adicionais podem ir no corpo do commit, em uma linha em branco após o título.
- Cada commit deve representar uma unidade lógica de trabalho — evitar misturar, por exemplo, uma correção de bug com a criação de um diagrama no mesmo commit.
- Usar o português para todas as mensagens, mantendo consistência com o restante da documentação do projeto.
- Evitar mensagens genéricas como "ajustes" ou "atualização"; preferir descrever o que de fato foi alterado.

3. Organização de pastas

Para manter a separação clara entre artefatos de documentação e código-fonte, o repositório segue a estrutura abaixo a partir da raiz do projeto, refletindo a organização já em uso:

/deu-branco-api	-> módulo Maven com o código-fonte da API (Spring Boot)
/deu-branco-api/src/main repository, service)	-> código de produção (config, controller, model,
/deu-branco-api/src/test	-> testes automatizados do módulo
/docs/Requisitos	-> documento de especificação de requisitos e diagramas
/docs/Diagramas	-> diagramas UML (classes, pacotes e sequência)
/docs/Padrões Adotados	-> documentação de padrões de projeto adotados
/docs/design	-> material de design (padrões visuais, referências)

Documentos como a especificação de requisitos, diagramas de classes e de pacotes, e a documentação de padrões adotados devem ser versionados dentro de /docs, nunca misturados com o código em /deu-branco-api. Isso mantém o histórico de commits do tipo docs separado, em termos de conteúdo, dos commits do tipo feat ou fix. Os testes automatizados não ficam em uma pasta isolada na raiz: seguem a convenção padrão do Maven, dentro de /deu-branco-api/src/test, ao lado do código que testam.

4. Resumo das responsabilidades da equipe

Integrante	Foco principal observado no histórico
João Vitor Reis	Back-end: API de autenticação, modelagem de domínio, diagramas de classes, regras de negócio (roles, questões).
Giovane Godoi	Front-end e documentação: design, diagrama de pacotes, README e documentação complementar.
Caio	Requisitos, especificação, padronização de processo e documentação de apoio, incluindo este guia de convenções de Git.